Comp 512
Spring 2012

Proliferation of DFA Problems

+ DEF-USE Chains

Copyright 2012, Keith D. Cooper & Linda Torczon, all rights reserved.

Students enrolled in Comp 512 at Rice University have explicit permission to make copies of these materials for their personal use.

Faculty from other educational institutions may use these materials for nonprofit educational purposes, provided this copyright notice is preserved.

The Round-robin Iterative Algorithm

$LIVEOUT(n_f) \leftarrow \emptyset$

for all other nodes n

  $LIVEIN(n) \leftarrow UEVAR(n)$

change ← true

while (change)

  change ← false

  for i ← 0 to N

    n ← rPreorder(i)

    $\text{TEMP} \leftarrow \bigcup_{s \in succ(n)} LIVEIN(s)$

    if $LIVEOUT(n) \neq \text{TEMP}$ then

      change ← true

      $LIVEOUT(n) \leftarrow \text{TEMP}$

      $LIVEIN(n) \leftarrow UEVAR(x) \cup (LIVEOUT(x) \cap \overline{VARKILL(x)})$

Meet operation

Transfer function

We could, for brevity, fold the transfer function into the computation of *LIVEOUT* and eliminate the need to represent *LIVEIN*.

We use this formulation for clarity.

The Worklist Iterative Algorithm

Worklist ← { all nodes, n_i }

$LIVEOUT(n_f) \leftarrow \emptyset$

for all other nodes *n*

  $LIVEIN(n) \leftarrow UEVAR(n)$

while (*Worklist* $\neq \emptyset$)

  remove a node *n* from *Worklist*

  $TEMP \leftarrow \cup_{s \in succ(n)} LIVEIN(s)$

  if $LIVEOUT(n) \neq TEMP$ then

    $LIVEOUT(n_i) \leftarrow TEMP$

    $LIVEIN(n_i) \leftarrow UEVAR(x) \cup (LIVEOUT(x) \cap \overline{VARKILL(x)})$

    $Worklist \leftarrow Worklist \cup preds(n)$

This algorithm avoids re-evaluating *LIVEOUT* when none of its constituents has changed. Thus, it does fewer set operations on *LIVEOUT* & *LIVEIN* than the round-robin algorithm. It does manipulate the *Worklist*, which is an added cost.

Implement the *Worklist* as a set – no duplicate entries

Round-robin Versus Worklist Algorithm

Termination, correctness, & complexity arguments are for the round-robin version

- The worklist algorithm does less work; it visits fewer nodes
- Consider a version with two worklists: *current* & *next*
  - → Draw nodes from *current* in RPO; add them to *next*
  - → Swap the worklists when *current* becomes empty (*1 r-r pass*)
  - → Makes same changes, in same order, as round robin
  - → Termination & correctness arguments should carry over
- Is this the best order? Maybe not …
  - → Iterate around a loop until it stabilizes
  - → Many data structures for worklist, many orders
  - → Your mileage will vary (a *measurable effect*)

Worklist algorithm is a <u>*sparse*</u> version of round-robin algorithm

GDFAP ≅ Global Data-Flow Analysis Problem

Proliferation of GDFAPs

In the late 1960's and the 1970's many data-flow problems were proposed

- GDFAP became the standard way to prove safety of a transformation
  - → New transformation implied new GDFAP
  - → Optimizing compilers spent a large fraction of compile time solving GDFAPs
  - → Computers were relatively slow(1 – 10 MIPS) and small (16 to 32 MB)
  - → Development of "frameworks" for GDFA

Live Variables

$|\text{LIVEOUT}| = |variables|$

Transformation: Eliminating unneeded stores

- *e* in a register, have seen last definition, never again used
- The store is <u>dead</u> (*except for debugging*)
- Compiler can eliminate the store

Standard $f(x) = a \cup (b \cap c)$. Single equation formulation

Data-flow problem: Live variables

$$\text{LIVEOUT}(b) = \cup_{s \in succ(b)} (\text{UEVAR}(s) \cup (\text{LIVE}(s) \cap \overline{\text{VARKILL}(s)}))$$

$$\text{LIVEOUT}(n_f) = \emptyset$$

- $\text{LIVE}(b)$ is the set of variables live on exit from b
- $\text{VARKILL}(b)$ is the set of variables that are redefined in b
- $\text{UEVAR}(b)$ is the set of variables used before any redefinition in b

Live analysis is a backward flow problem

LIVE plays an important role in both register allocation and the pruned-SSA construction.

Computing Available Expressions

$|AVAIL| = |expressions|$

An expression is <u>*available*</u> on entry to *b* if, along every path from the entry node to *b*, it has been computed & none of its constituent subexpressions has been subsequently modified.

$$AVAIL(b) = \cap_{x \in pred(b)} (DEEXPR(x) \cup (AVAIL(x) \cap \overline{EXPRKILL(x)}))$$

where

- *EXPRKILL(b)* is the set of expression killed in *b*, and
- *DEEXPR(b)* is the set of expressions defined in *b* and not subsequently killed in *b*

Initial conditions

$AVAIL(n_0) = \emptyset$, because nothing is computed before n_0

$AVAIL(n)$ = { all expressions }, because $\wedge$ is $\cap$

Available expressions is a forward data-flow problem

Using Available Expressions

AVAIL is the basis for *global common subexpression elimination*

- If $x+y \in AVAIL(b)$ then the compiler can replace any upwards exposed occurrence of $x+y$ in b with a reference to the previously computed value
- Standard caveats occur about preserving prior computations
  - → Hash $x+y$ and assign temporary names; copy all $x+y$ to name
  - → Walk backward through code to find minimal set of copies
  - → Other clever schemes abount
- Resulting algorithm is similar to value numbering
  - → Based on textual identity of expressions, not value identity
  - → Recognizes redundancy carried along loop-closing branch
- The classic & oldest form of GCSE (*Cocke, SIGPLAN Conference, 1970*)

Very Busy Expressions

An expression *e* is *very busy* on exit from *n*, *iff e* is evaluated & used along every path from *n* to n_f *and* evaluating *e* at the end of *n* would produce the same result as the next evaluation along those paths

The Plan

- Annotate each block *n* with a set *VERYBUSY(n)* that

  contains each expression
- Perform analysis to find opportunities
- If *e* is in *VERYBUSY(n)*, insert an evaluation at the end of *n* and eliminate the subsequent evaluations that it covers
  - → If *e* is in *VERYBUSY(n)* for successive blocks, want to insert it in the "right" *n*
  - → Might be the last *n* (minimize register demand) or least frequently executed *n* (minimize dynamic number of evaluations) or …
- This optimization aims, primarily, to reduce code space

Very Busy Expressions

|VERYBUSY| = |*expressions*|

Transformation: Hoisting

- *e* defined in <u>every</u> successor of *b*
- Evaluating *e* in *b* produces same result
- Saves code space, but shortens <u>no</u> path

Form of *f* is same as in *LIVE*

Data-flow problem: Very Busy Expressions

$$VERYBUSY(b) = \cap_{s \in succ(b)} (UEEXPR(s) \cup (VERYBUSY(s) \cap \overline{EXPRKILL(s)}))$$

$$VERYBUSY(n_f) = \emptyset$$

- *VERYBUSY*(b) contains expressions that are very busy at end of b
- *UEEXPR*(b) is the set of expressions used before they are killed in b
- *EXPRKILL*(b) is the set of expressions killed before they are used in b

VERYBUSY expressions is a backward flow problem

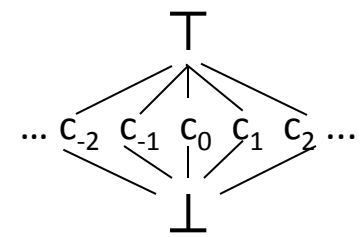
|*CONSTANTS*| = |*variables*|

Constant Propagation

(Classic formulation)

Transformation: Global Constant Folding

- Along every path to *p*, *v* has same known value
- Specialize computation at *p* based on *v*'s value



The Lattice *C*

Data-flow problem: Constant Propagation

Domain is the set of pairs $<v_i,c_i>$ where v_i is a variable and $c_i \in C$

$$CONSTANTS(b) = \wedge_{p \in preds(b)} f_p(CONSTANTS(p))$$

- $\wedge$ performs a pairwise meet on two sets of pairs
- $f_p(x)$ is a block specific function that models the effects of block p on the $<v_i,c_i>$ pairs in *x*

Form of *f* is quite different than in the other GDFAPs that we have seen

Constant propagation is a forward flow problem

*

Example

Meet operation requires more explanation

- $c_1 \wedge c_2 = c_1$ if $c_1 = c_2$, else $\perp$ (bottom & top as expected)

What about f_p ?

f_p does not fit into the mold of the functions in our Kam-Ullman rapid frameworks.

- If *p* has one statement then

  $x \leftarrow y$ with $CONSTANTS(p) = \{\ldots\langle x,l_1\rangle,\ldots\langle y,l_2\rangle\ldots\}$

  then $f_p(CONSTANTS(p)) = CONSTANTS(p) - \langle x,l_1\rangle + \langle x,l_2\rangle$

  $x \leftarrow y\ op\ z$ with $CONSTANTS(p) = \{\ldots\langle x,l_1\rangle,\ldots\langle y,l_2\rangle\ldots\ \rangle,\ldots\langle z,l_3\rangle\ldots\}$

  then $f_p(CONSTANTS(p)) = CONSTANTS(p) - \langle x,l_1\rangle + \langle x,l_2\ op\ l_3\rangle$

- If *p* has *n* statements then

  $f_p(CONSTANTS(p)) = f_n(f_{n-1}(f_{n-2}(\ldots f_2(f_1(CONSTANTS(p)))\ldots)))$

  where f_i is the function generated by the ith statement in *p*

Constant propagation, in its more general forms, can become intractable because it encodes arithmetic.

GDFAP ≅ Global Data-Flow Analysis Problem

Proliferation of GDFAPs

In the late 1960's and the 1970's many data-flow problems were proposed

- GDFAP became the standard way to prove safety of a transformation
  - → New transformation implied new GDFAP
  - → Optimizing compilers spent a large fraction of compile time solving GDFAPs
  - → Computers were relatively slow(1 – 10 MIPS) and small (16 to 32 MB)
  - → Development of "frameworks" for GDFA
- As transformations proliferated, need for a new paradigm emerged
  - → One GDFAP that could be used for multiple transformations
  - → Simplify the implementation
  - → Reduce the time spent in analysis
  - → The result was the development of information chains

Like all facile explanations, this one is not universal. The adoption of information chains was spread over many years and did not proceed at a uniform rate.

Information Chains

A tuple that connects 2 data-flow events is a *chain*

- Chains express data-flow relationships directly
- Chains provide a graphical representation
- Chains jump across unrelated code, simplifying search

event ≅ definition or use

We can build chains efficiently

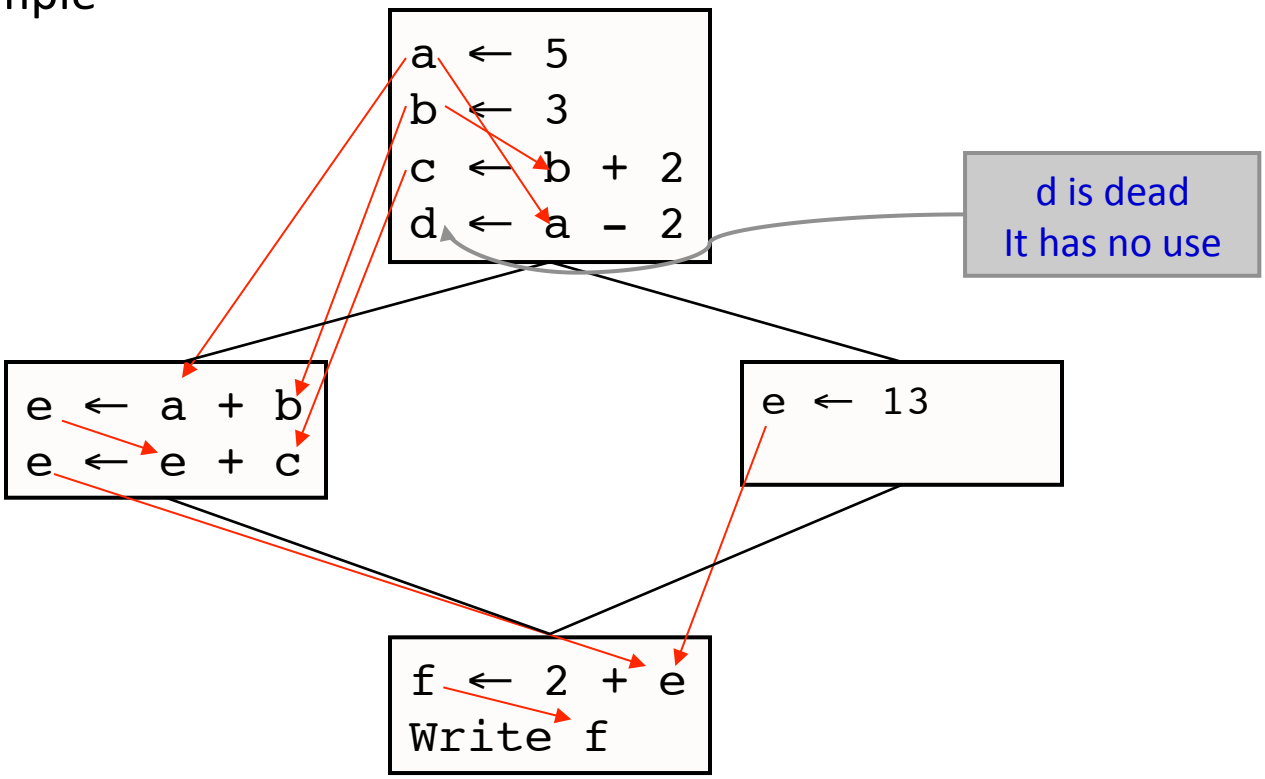
Four interesting types of chain

Source	Sink	Dependence Type
DEF	USE	*true, flow*
USE	DEF	*anti*
DEF	DEF	*output*
USE	USE	*input*

DEF-USE chains are the most common

Information Chains

Example



DEF-USE Chains

DEF-USE Chains form a sparse evaluation graph that we can use in many transformations.

For example, a DEF with no reachable use is *dead.*

Notation

Assume that, ∀ operation *i* and each variable *v,*

- *DEFS(v,i)* is the set of operations that may have defined *v* most recently before *i*, along some path in the CFG
- *USES(v,i)* is the set of operations that may use the value of *v* computed at *i*, along some path in the CFG

$$x \in DEFS(A,y) \Leftrightarrow y \in USES(A,x)$$

To construct DEF-USE chains, we solve *reaching definitions* (*YAGDFAP*)

- A definition *d* of some variable *v* <u>*reaches*</u> an operation *i* if and only if *i* reads *v* and there is a <u>*v-clear*</u> path from *d* to *i*

  → *v-clear* ⇒ no definition of *v* on the path

- Prior definition of *v* in same block ⇒ $|DEFS(v,i)| = 1$
- No prior definition ⇒ $|DEFS(v,i)| \geq 1$

Domain is |definitions|, same as number of operations

Reaching Definitions

The equations

$$REACHES(n) = \emptyset, \forall\ n \in N$$

$$REACHES(n) = \cup_{p \in preds(n)} (DEDEF(p) \cup (REACHES(p) \cap \overline{DEFKILL(p)}))$$

Form of f is same as in LIVE

- *REACHES(n)* is the set of definitions that reach block *n*
- *DEDEF(N)* is the set of definitions in *n* that reach the end of *n*
- *DEFKILL(n)* is the set of defs obscured by a new def in *n*

Computing *REACHES(n)*

- Use any data-flow method (*rapid framework*)
- Zadeck shows a simple linear-time algorithm

F.K. Zadeck, "Incremental data-flow analysis in a structured program editor," *Proceedings of the SIGPLAN 84 Conf. on Compiler Construction*, June, 1984, pages 132-143.

Building DEFS sets

The Plan

1. Find basic blocks & build the CFG
2. ∀ block *b,* compute *REACHES(b)*

3. ∀ block *b*, ∀ operation *i*, ∀ referenced name *v*,

   Set *DEFS(v,i)* according to the earlier rule

   If there is a prior definition, *d*, of *v* in *b*

   $DEFS(v,i) \leftarrow d$

   Otherwise

   $DEFS(v,i) \leftarrow \{d \mid d \text{ defines } v \ \& \ d \in REACHES(b)\}$

To build USES

- Invert *DEFS*, or
- Solve *reachable uses*, and use the analogous construction

Building DEF-USE Chains

Miscellany

- Domain of *REACHES* is the set of definitions ($\propto$ |operations|)
- Domain of *DEFS* & *USES* is also definitions
- Need a compact representation of *DEFS* & *USES*

Detecting Anomalies

- *DEFS(v,i)* = Ø implies use of a <u>never initialized</u> variable
- Add a definition for each *v* to n_0 to detect larger set of anomalies
  - → If initial def ∈ *DEFS(v,i)* then ∃ a path to *i* with no initialization

And, how do we use these information chains? *Next lecture …*

*